SYNERGETICS
GET IT RIGHT

Java 17 New Features

Makarand Bhoir

Microsoft Partner | Silver Learning Gold Cloud Platform

Microsoft Cloud Solution Provider

Educate | Advise | Implement | Manage

Agenda

- sealed classes
- Records
- switch expressions and pattern matching
- text blocks

Sealed classes

- Inheritance enables any class to inherit from any other class.
- Making a class final prevents any class from inheriting from that class i.e., the final class cannot become a super type at all.
- What if you wanted your class to be available for inheritance but only to certain classes?

Sealed classes

- Sealed classes enable us to control the scope of inheritance by enabling us to specify a classes' subtypes.
- Also works with interfaces (we can define what classes implement the interface).

records

- Records are a special type of class that help avoid boilerplate code. They are considered “data carriers”.
- Records are immutable and are final by default.
- You cannot extend your custom record because records already (implicitly) extend from the Record class. This is similar to enums (which implicitly extend from Enum).

records

- Records can have both static fields and static methods.
- Records can have instance methods.
- Records cannot have instance fields. All the instance fields are listed as “components” in the record declaration.
- Records can implement interfaces.

records

- Records are specified using a record declaration where you specify the “components” of the record.
- Implicitly generated are:
  - canonical constructor
  - toString () the string representation of all the record class's
  - components , with their names
  - equals() and hashCode() which specify that two record classes are equal if they are of the same type and contain equal component values
  - public accessor methods with the same name as the components.

records

- You can override all the default implementations. This includes the canonical constructor (perhaps for data validation).
- Compact constructor is a concise variation of the canonical constructor and is specific to records.
-

```
// default canonical constructor
3 usages
public CarRecord(String regNumber, String owner)  {
    this.regNumber=regNumber;
    this.owner     =owner;
}
```

```
// custom canonical constructor
3 usages
public CarRecord(String regNumber, String owner)  {
    if(regNumber.length() <= 4){
        throw new IllegalArgumentException();
    }
    this.regNumber=regNumber;
    this.owner     =owner;
}
```

records

```
public record CarRecord(String regNumber, String owner)  {}
```

```
// custom canonical constructor
3 usages
public CarRecord(String regNumber, String owner)  {
    if(regNumber.length() <= 4){
        throw new IllegalArgumentException();
    }
    this.regNumber=regNumber;
    this.owner     =owner;
}
```

```
// compact constructor - specific to records
3 usages
public CarRecord  {
    if(regNumber.length() <= 4){
        throw new IllegalArgumentException();
    }
}
```

Pattern matching

SYNERGETICS
GET IT RIGHT

- The switch statement has undergone several changes over the years.
- switch expressions were introduced as a “preview feature” in Java 12 and became permanent in Java 14.
- a “preview feature” is a feature that is designed and implemented but not yet permanent (and may never be); it allows a large developer audience to test out the feature before committing to it.
- yield statement introduced in Java 13 to support switch expressions.

Pattern matching

- This is a “preview feature” in Java 17.
- Preview features are, by default disabled so you must explicitly enable them.

Text Block

- A text block is a String object and as a result, shares the same properties as String objects (immutable and interned).
- you can call String methods on a text block.
- A text block begins with three double quote characters followed by newline
  - text blocks cannot be put on one line

Text Block Example

- TextBlockExample.java

SYNERGETICS

GET IT RIGHT

Thank You

Educate | Advise | Implement | Manage